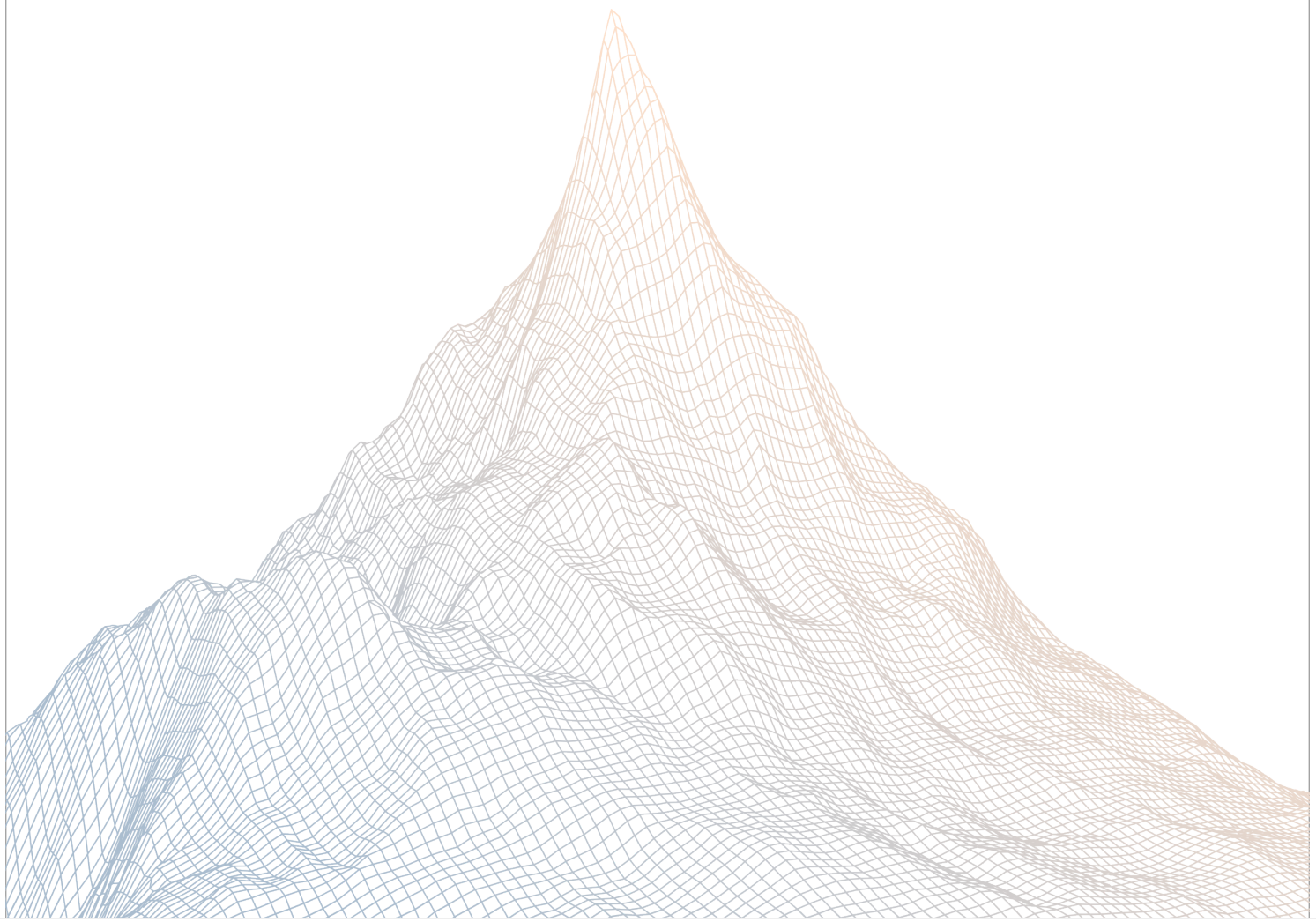
Zenith

GMX

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: September 10th to 14th, 2026

AUDITED BY: Mario Poneder
peakbolt

Contents

1 Introduction 3

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 4

2.1 About GMX 4

2.2 Scope 4

2.3 Audit Timeline 5

2.4 Issues Found 5

3 Findings Summary 6

4 Findings 7

4.1 Low Risk 7

4.2 Informational 26

1 Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About GMX

GMX Solana is a decentralized spot and perpetual exchange that supports low swap fees and low price impact trades.

Trading is supported by unique multi-asset pools that earns liquidity providers fees from market making, swap fees and leverage trading.

Dynamic pricing is supported by Chainlink Oracles and an aggregate of prices from leading volume exchanges.

2.2 Scope

The engagement involved a review of the following targets:

Target	gmx-solana
Repository	https://github.com/gmsol-labs/gmx-solana
Commit Hash	`eaa69c85b39dfeb30058845765a1f612ae20eca0`
Files	`crates/**/*.rs – excl. cli, decode, programs, sdk, solana-utils, test folders, and inline test code.` `programs/**/*.rs – excl. test folders, and inline test code.`

Target	gmx-solana Mitigation Review
Repository	https://github.com/gmsol-labs/gmx-solana
Commit Hash	`8af1e70c0c0b6ff9d6b41ba4997910588fbcec28`
Files	`crates/**/*.rs – excl. cli, decode, programs, sdk, solana-utils, test folders, and inline test code.` `programs/**/*.rs – excl. test folders, and inline test code.`

2.3 Audit Timeline

September 10, 2026	Audit start
September 14, 2026	Audit end
October 9, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	9
Informational	4
Total Issues	**13**

3

Findings Summary

ID	Description	Status
L-1	Increase-position balance validation omits the outgoing builder fee	Resolved
L-2	Builder fee checkpoints do not advance the order's oracle freshness boundary	Resolved
L-3	Read-only owner metadata can block GT exchange completion	Resolved
L-4	Fractional fees rounded out of collection still contribute to trading GT rewards	Acknowledged
L-5	Market initialization uses the fee-receiver default for reserve limits	Resolved
L-6	Delayed builder-fee authorization can apply to a recreated order	Resolved
L-7	Keepers can cancel orders and collect execution fees by supplying malformed accounts	Acknowledged
L-8	Builder fee collection excludes same-mint secondary output	Resolved
L-9	Revoking a builder fee unnecessarily requires a claim ATA	Resolved
I-1	Pending builder fee revocation is unavailable while fee attachment is disabled	Resolved
I-2	Increase trade events leave the builder fee output mint unset	Resolved
I-3	Builder claim vaults lack a rent recovery path	Resolved
I-4	Deferred order closing permits retroactive referral rewards	Acknowledged

4

Findings

4.1 Low Risk

A total of 9 low risk findings were identified.

[L-1] Increase-position balance validation omits the outgoing builder fee

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- programs/store/src/ops/order.rs#L1763-L1860
- programs/store/src/states/market/utils.rs#L31-L131

Description:

The scoped diff introduces a builder-fee outflow for increase orders without including it in the existing market-balance validation.

Gross collateral enters the market's recorded bank balance. The fee is deducted from the collateral increment passed to the position model and scheduled for transfer into the order's escrow. However, validation excludes only funding withdrawals:

```
transfer_out.transfer_out(false, payable_amount)?;

// Later, after the position update:
position.market().validate_market_balances(
 long_amount
 .try_into()
 .map_err(|_| error!(CoreError::TokenAmountOverflow))?,
 short_amount
 .try_into()
 .map_err(|_| error!(CoreError::TokenAmountOverflow))?,
)?;
```

The accompanying comment incorrectly states that the fee never enters market balances. It is excluded from position collateral but remains in the bank ledger until transfer-out, which subtracts it without repeating the minimum-balance check.

Let `B` be the bank balance before outgoing transfers, `K` the funding withdrawal in the fee mint, and `F` the builder fee. Validation checks `B − K ≥ R`, although completion requires `B − K − F ≥ R`. Here, `R` is the greater of the pool/fee floor and aggregate collateral floor, since these are checked separately.

A potential violation requires an existing position with claimable funding, a positive authorized builder fee, and headroom satisfying `0 ≤ B − K − R < F`. Funding can accrue before paying positions settle, providing a possible mechanism for consuming that headroom. Sufficient shared-vault liquidity does not enforce the market-local floor.

Recommendations:

It is recommended to compute separate validation exclusions from the scheduled funding amounts, add the builder fee once to the collateral mint's exclusion using checked arithmetic, and validate the current market before committing the position update. Preserve aggregation when long and short tokens share a mint.

Keep actual funding payouts unchanged. The `total_long_token_amount()` and `total_short_token_amount()` helpers exclude `final_output_token`, so using those helpers alone is insufficient. Correct the comment to distinguish net position collateral from the gross bank balance.

GMX: Resolved with PR-450.

Zenith: Verified.

[L-2] Builder fee checkpoints do not advance the order's oracle freshness boundary

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- programs/store/src/instructions/builder_fee.rs#L409-L424
- programs/store/src/ops/order.rs#L1488-L1584
- programs/store/src/ops/order.rs#L1901-L1924
- programs/store/src/ops/order.rs#L2080-L2127

Description:

The builder fee checkpoint instruction introduced within the scoped diff allows an order owner to change a pending position order's builder fee factor without advancing its oracle freshness boundary. The underlying setter updates only the builder and factor:

```
pub(crate) fn set_builder_fee(&mut self, builder: Pubkey, factor: u128) {
 self.builder = builder;
 self.builder_fee_factor = factor;
}
```

Execution derives its minimum acceptable oracle timestamp and slot from the existing order header and applicable position and validity constraints. Consequently, authentic price observations published before the fee change can remain admissible when execution uses the newly selected factor.

For a partial decrease, the program adds an estimated builder fee to the requested collateral withdrawal. The resulting output can pass through the order's previously configured receive swap, and the builder fee is subsequently allocated from the final output token. The checkpoint therefore changes the amount exposed to that conversion; it does not select a new token or swap route.

A conditional value-transfer scenario follows:

1. An owner has a pending, sufficiently collateralized partial decrease with nonzero size, zero ordinary collateral withdrawal, and a configured collateral-token-to-output-token swap.

2. After an admissible oracle observation is published and relative prices move favorably, the owner checkpoints a positive fee to their own eligible builder account.
3. An authorized keeper executes using that earlier observation. The increased withdrawal supplies additional collateral to the existing swap, and the owner can recover the builder allocation in the output token through settlement and claiming.

The additional conversion can disadvantage liquidity providers if its favorable price difference exceeds swap fees, price impact, position-related costs, and other execution costs. This establishes a conditional incremental conversion advantage, not guaranteed net profit.

Reachability requires enabled builder fees, a positive configured cap, suitable escrow and claim accounts, sufficient remaining collateral, admissible earlier oracle data, and compatible keeper execution. An execution transaction that also attempts to close the order before settling its newly recorded fee reverts. Same-token self-payment alone does not establish this advantage; full-close behavior or withdrawal suppression can also eliminate the additional-volume effect. Deployment configuration and actual keeper behavior have not been established.

Self-checkpointing and pending fee replacement are intentional features. The concern is the missing freshness boundary for their effect on conversion volume. The scenario depends on an owner's update preceding otherwise legitimate keeper execution; assuming malicious keeper price selection would weaken its distinction from documented keeper trust assumptions. The reviewed documentation does not explicitly accept this specific pricing consequence.

Recommendations:

It is recommended to record a checkpoint timestamp and slot whenever the fee factor actually changes, and incorporate them into the execution oracle's minimum bounds while preserving existing order, position, and validity constraints. Identical-factor checkpoints should not unnecessarily invalidate prices.

Preserve the original market-order expiration unless extending it is an explicit design decision. Define handling when the checkpoint boundary exceeds that expiration, and preserve existing cancellation and zero-factor revocation paths. Any additional state should maintain account layout and backward compatibility. Define same-slot ordering explicitly: inclusive timestamp and slot comparisons alone do not guarantee publication strictly after the change.

Keeper notifications and refreshed SDK hints can support this behavior. If an expected factor or revision guard is added, enforce it on-chain; client-side checks alone cannot prevent changes between transaction construction and execution.

GMX: Resolved with PR-466.

Zenith: Resolved. The PR fixes the missing freshness update, with a documented same-slot limitation and a restriction on market-order revocation.

[L-3] Read-only owner metadata can block GT exchange completion

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- programs/treasury/src/instructions/gt_bank.rs#L270-L272
- programs/treasury/src/instructions/gt_bank.rs#L323-L330
- programs/store/src/instructions/gt.rs#L378-L404

Description:

Treasury's `complete_gt_exchange` instruction declares the exchange owner as a signer without marking it writable. Generated account metadata and the checked-in IDL consequently omit writable permission. Completion nevertheless invokes Store's `close_gt_exchange`, which requires a writable owner because closing the exchange account returns its rent to that owner.

```
// Treasury: CompleteGtExchange
pub owner: Signer<'info>,

// Store: CloseGtExchange
#[account(mut)]
pub owner: UncheckedAccount<'info>,
```

An otherwise valid completion transaction fails when a separate sponsor pays its fee and the owner remains read-only. Treasury's CPI requests writable permission that it did not receive, causing the runtime to reject the invocation with `PrivilegeEscalation`. Smart-wallet wrappers forwarding a read-only owner encounter the same mismatch. A signature, including valid PDA signing, does not grant writable permission.

The missing annotation does not prohibit callers from supplying a writable owner. Another instruction or account alias can supply that permission in a top-level transaction. The native SDK also avoids this failure by setting `owner = self.payer()`, since the fee payer is writable. No on-chain requirement that the owner pay the transaction fee was identified.

The failing CPI occurs before exchange deletion, payout and bank-accounting updates. Affected integrations cannot complete the claim until they correct the permissions, but the exchange and reserved funds remain available for retry. Failed transactions can incur trans-

action fees. Unauthorized privilege elevation and permanent asset loss are not established

Recommendations:

It is recommended to mark Treasury's owner signer writable, regenerate the IDL and generated account metadata, and retain the signature requirement:

```
/// Owner.
#[account(mut)]
pub owner: Signer<'info>,
```

Clients can work around the reviewed implementation by explicitly passing the owner as writable. Smart-wallet wrappers must receive and forward writable permission through every relevant CPI boundary.

GMX: Resolved with PR-452.

Zenith: Verified.

[L-4] Fractional fees rounded out of collection still contribute to trading GT rewards

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- crates/model/src/params/fee.rs#L104-L139
- crates/model/src/params/fee.rs#L847-L861
- programs/store/src/states/order.rs#L691-L726

Description:

Order and borrowing fees are calculated in USD value and rounded down when converted into collateral token units. However, `paid_order_and_borrowing_fee_value`, which determines trading GT rewards, retains the nominal value before this conversion.

The borrowing calculation illustrates the discrepancy:

```
let amount = value
 .checked_div(price)
 .ok_or(crate::Error::Computation("calculating borrowing amount"))?;
let paid_fee_value = self
 .paid_order_and_borrowing_fee_value
 .checked_add(&value)
 .ok_or(crate::Error::Computation("adding borrowing fee value"))?;
```

Order fees similarly retain their nominal value. On an otherwise valid decrease where rounded fees total zero token units, fee collection is skipped without clearing the retained value. Successful execution in a GT-enabled market forwards this value into the user's cumulative fee credit. It can mint GT immediately or remain as credit toward subsequent minting. Increases also retain the nominal value while charging rounded token amounts.

For example, a $0.0005 decrease from a healthy 100 position at a 0.1% effective order fee produces a nominal fee of $0.0000005. With six-decimal collateral priced at 1, collection rounds to zero token units. Assuming unchanged prices, zero PnL, impact, funding and borrowing fees, compatible risk settings, and successful keeper execution, this value still reaches GT accounting. With no carried remainder and a minting cost of $0.0000001 per GT base unit, it produces five GT base units. This minting cost is an illustrative configuration.

For ordinary collection at collateral atomic-unit price `P`, the excess credited value is:

```
(nominal_order_fee mod P) + (nominal_borrowing_fee mod P) < 2P
```

For the example collateral, this is below $0.000002 per combined settlement. Fractional overstatement also occurs alongside nonzero collection.

The resulting GT credit exceeds the corresponding collected fee value. Transaction and execution costs remain, and material Treasury loss or profitable repetition has not been established. The mechanism requires ordinary keeper execution and compatible configuration without requiring dishonest keepers or manipulated prices.

Recommendations:

It is recommended to derive GT-eligible paid fee value in the shared fee accounting from the rounded, actually collected order and borrowing amounts, using the same collateral minimum price used for conversion.

For fully collected eligible fees, calculate:

```
(order_pool_amount + order_receiver_amount + borrowing_total_amount) × collateral_min_price
```

Use checked arithmetic and preserve existing rules that clear credit when eligible fees were not collected. Apply the calculation consistently to increases and decreases.

Keep funding, liquidation, and builder fees excluded. Generic total-fee helpers can include liquidation fees and therefore require explicit filtering. Clearing credit only when the total token charge is zero would leave fractional overstatement on nonzero charges.

GMX: Acknowledged. This will be fixed in the future version.

Zenith: Acknowledged.

[L-5] Market initialization uses the fee-receiver default for reserve limits

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- programs/store/src/states/market/config.rs#L174-L175
- crates/model/src/market/base.rs#L371-L386
- crates/model/src/action/swap.rs#L251-L257
- crates/model/src/action/withdraw.rs#L228-L234

Description:

`MarketConfig::init` initializes the reserve factor using the default intended for fee receivers:

```
self.reserve_factor = constants::DEFAULT_RECEIVER_FACTOR;
```

`DEFAULT_RECEIVER_FACTOR` is 70%, whereas the dedicated `DEFAULT_RESERVE_FACTOR` is 50%. The supplied example market configuration also explicitly selects 50%. Market initialization persists the incorrect default, and reserve validation subsequently uses it to calculate the maximum reserved exposure relative to pool value.

The error weakens the reserve constraint applied after swaps and withdrawals. For unchanged reserved exposure `R`, this check permits remaining pool value down to `R / 0.70`, instead of the `R / 0.50` required by the dedicated default. Consequently, liquidity-reducing operations can pass with utilization above 50% and up to 70%, provided their other validations also pass.

Position increases additionally enforce the open-interest reserve factor, which defaults to 45% and uses the same reserved-value calculation. Their effective default limit therefore remains 45%. Swaps and withdrawals do not repeat that check, so the weaker general reserve constraint remains relevant. Separate pool and PnL validations can restrict these operations before they reach 70%.

Exposure requires an enabled market to retain the initialized value and reach utilization where the thresholds produce different outcomes. Market creation and configuration updates are privileged. The example configuration starts disabled and supplies the correct reserve factor, so applying it before activation avoids the defect. However, neither the program nor the SDK creation helper requires that override before enabling a market.

The demonstrated impact is additional permitted liquidity removal and a weaker reserve buffer. An explicitly selected 70% parameter may be a legitimate administrative policy; this issue concerns its unintended selection from an unrelated default.

Recommendations:

It is recommended to initialize the reserve factor using its dedicated constant:

```
self.reserve_factor = constants::DEFAULT_RESERVE_FACTOR;
```

Review existing persisted configurations separately, since changing initialization affects only subsequently created markets. Correct unintended defaults through authorized configuration updates while preserving deliberate administrative settings.

Before lowering an existing market's factor, assess utilization on both sides. The tighter constraint may restrict swaps and withdrawals until exposure decreases or liquidity increases.

GMX: Resolved with PR-451.

Zenith: Verified.

[L-6] Delayed builder-fee authorization can apply to a recreated order

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- programs/store/src/instructions/builder_fee.rs#L184-L201
- programs/store/src/instructions/exchange/order.rs#L209-L217
- programs/store/src/ops/order.rs#L189-L207

Description:

When a builder-fee signature is intended to authorize only the current logical order, reusing that order's nonce can cause the signature to authorize a replacement order instead. This finding depends on that per-instance authorization contract; an explicitly address-wide authorization lasting until transaction expiry would permit this behavior.

The owner-signed `set_builder_fee` instruction introduced in the audited diff binds authorization to an Order address and an expected advertised factor. It does not bind the signature to the order's market or creation instance.

Order creation derives its address from a caller-supplied nonce:

```
seeds = [Order::SEED, store.key().as_ref(), owner.key().as_ref(), &nonce],
```

After successful closure, the same nonce can initialize another order at that address. Creation assigns a new market-scoped order ID, but checkpointing does not compare that ID against a signed expectation.

The following sequence is therefore possible:

1. The owner signs a builder-fee checkpoint for a pending order. A submitter retains the fully signed transaction without executing it.
2. The owner successfully closes the order with zero builder-fee debt, then creates a replacement using the same Store, owner, and nonce.
3. While the withheld transaction remains valid, the submitter delivers its unchanged bytes. The replacement passes checkpoint validation if it remains pending, retains the signed

fee mint, and satisfies the advertised-rate, cap, feature, order-kind, and escrow requirements.

4. Execution charges the replacement using the old authorization, and settlement pays the builder.

For example, a 1% checkpoint prepared for a $1,000 increase can attach to a replacement $100,000 decrease, producing a $1,000 builder fee when output covers it. The percentage remains authorized; the concern is applying consent to a different logical order.

The owner or integration must perform the closure and nonce reuse; the submitter cannot independently create a replacement for an unwilling owner. Fresh SDK nonce defaults mitigate accidental reuse, but explicit selection is supported and uniqueness is not enforced on-chain. No deployed automatic reuse workflow was established.

The withheld transaction executes for the first time. Protection against resubmitting an already executed transaction remains intact.

Recommendations:

It is recommended to include the expected `header.market` and `header.id` in the signed checkpoint arguments and validate both before updating the checkpoint.

The market must accompany the order ID because its counter is market-scoped. Any alternative generation identifier must distinguish successive creations. Previous builder/factor values or a revision that resets on recreation are insufficient.

Make generation validation mandatory for the protected checkpoint interface, update its SDK and IDL representations, and avoid retaining an unguarded path that permits the same operation.

Clients should avoid nonce reuse until outstanding signatures targeting that address have expired or been invalidated, and explicitly define authorization lifetime.

GMX: Partially Resolved with PR-462 and we accept the risk that `set_builder_fee` could potentially be replayed against an order with the same nonce in the same market. As the comment says, discipline at the builder level mitigates this to some extent, and it is not worth spending much effort trying to solve it. We also believe `update_order` has a similar problem, so if we were to fix it, we would consider a more systematic approach, such as enforcing nonce monotonicity.

Zenith: Resolved. Fixed for cross-market, while same-market is mitigated at the builder level.

[L-7] Keepers can cancel orders and collect execution fees by supplying malformed accounts

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- order.rs#960

Description:

`ExecuteOrderOperation::execute` converts selected economic execution failures into successful order cancellations.

However, its catch-all branch also converts malformed remaining-account data and other keeper-controlled setup errors into cancellations.

An `ORDER_KEEPER` can cancel a user's order and collect its execution fee by supplying an invalid trailing account instead of allowing the transaction to revert.

Example Scenario

1. A user creates an order with 100 collateral tokens and prepays an execution fee of 0.01 SOL.
2. An `ORDER_KEEPER` supplies all required feed and market accounts, then appends the System Program where a `VirtualInventory` account is expected.
3. `RemainingAccountsForMarket::new` fails to decode the trailing account, but `should_throw_error` remains false and the catch-all branch returns a failed `TransferOut` successfully.
4. The outer handler cancels and refunds the order, then pays the 0.01 SOL execution fee to the keeper.

```
Err(err) if !(should_throw_error || self.throw_on_execution_error) => {
 msg!("Execute order error: {}", err);
 Ok((Default::default(), false))
}
```

Recommendations:

Move the existing remaining-account preparation and validation outside the error-handling block that converts execution failures into cancellations. `RemainingAccountsForMarket::new` already validates the supplied account types; it does not need to be called a second time.

Perform that parsing and the subsequent account loading and association checks before entering the cancellation-catching block, propagating failures directly with `?`. Pass the validated execution context into `perform_execution` and reuse it rather than reparsing the accounts there.

This makes an invalid keeper-supplied account revert the transaction, preserving the pending order and rolling back keeper payment. Merely moving `RemainingAccountsForMarket::new` is insufficient if later account-loading or association failures remain inside the broad catch.

As additional hardening, restrict cancellation to explicitly approved economic failures and propagate configuration, account-validation, and unexpected errors. This protects against setup errors that might otherwise remain in, or later be introduced into, the execution path.

GMX: Acknowledged. This will be fixed in the future version.

Zenith: Acknowledged by client.

[L-8] Builder fee collection excludes same-mint secondary output

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- order.rs#2010

Description:

`execute_decrease_position` caps the recorded builder fee at the final primary output amount after receive-token routing. It does not include `secondary_output_amount`, even when the secondary mint equals the final-output mint and both outputs are transferred into the same order-owned associated token account.

Builder-fee underpayment is intentionally tolerated when a decrease has insufficient output funds, so that a fee shortfall does not prevent a position from closing. However, excluding same-mint secondary output understates the funds available for the fee: builders can receive less than the calculated fee even when the combined outputs in the fee token are sufficient.

```
let paid_amount =
 clamp_builder_fee_amount(payable_amount, output_amount.into());
```

Routing Conditions

1. A position decrease produces collateral and profit outputs in different tokens, so the initial same-token merge keeps them separate.
2. Subsequent receive-token routing converts the primary collateral output into the secondary profit token.
3. `revertible_swap` returns the two output amounts separately even though their final mints now match. Its balance validation aggregates matching-token amounts for validation, but does not merge the returned amounts.
4. The builder fee remains capped at the primary output alone. This reduces the recorded fee only when the primary output is below the calculated fee and the matching secondary output could cover some or all of the difference.

Outputs whose tokens already match during the initial decrease processing are merged there and do not exhibit this specific condition. The distinction is that the outputs become the same token only during later receive-token routing.

Example Scenario

1. A decrease produces collateral and profit outputs in different tokens, with the profit output consisting of 100 USDC.
2. Receive-token routing converts the primary collateral output into 50 USDC. The two returned amounts remain separately tracked as 50 USDC primary output and 100 USDC secondary output.
3. Both outputs are transferred into the same order-owned USDC escrow, providing a total of 150 USDC.
4. The calculated builder fee is 120 USDC, but collection is capped at the 50 USDC primary output. Only 50 USDC is recorded for the builder, leaving 70 USDC of the calculated fee unpaid despite sufficient USDC in the combined outputs.

Recommendations:

Include secondary output in fee availability when its mint matches the final output token, then clamp the calculated fee against that combined amount. Preserve the existing partial-payment behavior when matching output funds are genuinely insufficient. This addresses the undercollection without restricting the final output token to collateral or requiring all decreases to pay the full fee.

```
let available_amount = if final_output_token == secondary_output_token {
 u128::from(output_amount)
 .checked_add(u128::from(secondary_output_amount))
 .ok_or_else(|| error!(CoreError::TokenAmountOverflow))?
} else {
 u128::from(output_amount)
};

let paid_amount = clamp_builder_fee_amount(payable_amount,
 available_amount);

// Combined availability may exceed u64; retain checked conversion.
let recorded_amount =
 u64::try_from(paid_amount).map_err(|_|
 error!(CoreError::TokenAmountOverflow))?;
order.record_builder_fee(recorded_amount)?;
```

Keep the output transfers unchanged, since settlement subsequently deducts the recorded fee from escrow. Update the surrounding comments to describe the combined availability and remove the assumption that the recorded amount is bounded by a single u64 output.

Validate that matching outputs cover the full fee when their combined amount is sufficient, different-mint secondary output remains excluded, genuine shortfalls retain partial payment, and amounts exceeding the recorded field's u64 range are handled explicitly.

GMX: Resolved with PR-458.

Zenith: Verified.

[L-9] Revoking a builder fee unnecessarily requires a claim ATA

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- builder_fee.rs#229

Description:

`set_builder_fee` lets an order owner replace a pending order's builder checkpoint, with a zero advertised factor serving as the documented revocation path.

However, the instruction always requires the replacement builder's final-output-token claim ATA, even when the replacement factor is zero and no fee can ever be transferred to that vault.

Owners cannot revoke an existing builder fee without first creating and funding an otherwise-unused PDA-owned ATA, adding an avoidable rent and account-availability dependency to a de-risking operation.

Example Scenario

1. An owner has a pending order with a nonzero builder fee and prepares its own zero-factor User Account to revoke the fee.
2. The owner's User Account has no ATA for the order's final-output mint because it has never received builder fees in that token.
3. The owner submits `set_builder_fee` with `expected_factor = 0`.
4. Anchor rejects the missing mandatory `claim_vault`, so the owner must create and rent-fund an unused ATA before the fee can be revoked.

```
#[account(
 associated_token::mint = final_output_token,
 associated_token::authority = builder,
)]
pub claim_vault: Box<Account<'info, TokenAccount>>,
```

Recommendations:

Update the existing `set_builder_fee` instruction instead of adding a dedicated clearing instruction. Make `claim_vault` optional, retaining its associated-token constraints whenever an account is supplied:

```
#[account(
 associated_token::mint = final_output_token,
 associated_token::authority = builder,
)]
pub claim_vault: Option<Box<Account<'info, TokenAccount|>,
```

After checking that the builder's advertised factor matches `expected_factor`, require `claim_vault.is_some()` whenever that validated factor is nonzero. A nonzero attachment must still provide the correct initialized claim ATA to preserve settlement safety. A zero-factor revocation may pass `None`, avoiding creation and rent funding of an unused ATA.

Preserve the owner-signature, pending-order, advertised-factor matching, and other existing authorization checks. Update the SDK and IDL for the optional account so revocation can omit the vault. Add coverage showing that revocation using the owner's zero-factor User Account succeeds without a claim ATA, nonzero attachment without one fails, and a supplied vault must still satisfy the ATA constraints.

GMX: Resolved with PR-454.

Zenith: Verified.

4.2 Informational

A total of 4 informational findings were identified.

[I-1] Pending builder fee revocation is unavailable while fee attachment is disabled

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- programs/store/src/instructions/builder_fee.rs#L286-L305
- programs/store/src/ops/order.rs#L1138-L1148
- programs/store/src/lib.rs#L3174-L3193

Description:

Revoking a builder fee on a pending order requires calling `set_builder_fee` with a User Account advertising zero. This shares the attachment entrypoint and its unconditional feature check:

```
ctx.accounts
 .store
 .load()?
 .validate_not_restarted()?
 .validate_feature_enabled(
 DomainDisabledFlag::BuilderFee,
 ActionDisabledFlag::Default,
 )?;
```

If a privileged feature keeper disables `BuilderFee:Default` after an owner attaches a nonzero fee, the owner can no longer revoke that authorization through this instruction. Execution continues to use the saved factor when the order's separate execution gate and other conditions permit it.

The public API documents that replacing a checkpoint reruns every validation and requires BuilderFee to remain enabled. Preserving execution and settlement of previously authorized fees is also deliberate. This is therefore an informational policy observation: disabling at-

tachment also removes revocation access, although revoking an unexecuted authorization would not erase a recorded fee debt.

The owner can generally cancel and recreate the order when the independent cancellation and creation conditions permit it. Recreation changes order identity and timing and incurs transaction costs. The continued fee remains the one previously authorized by the owner.

Restart recovery further constrains availability under an existing protocol policy. An unacknowledged cluster restart blocks owner revocation and pending-order cancellation, while ordinary keepers also fail authorization. Execution during that interval requires `RESTART_ADMIN` and remains subject to execution checks. This is supporting context rather than a newly introduced restart defect.

Lowering the fee cap alone does not prevent zero-factor revocation. Even when enabled, the current revocation path requires an existing claim-vault ATA for the zero-advertising User Account, which can require an additional rent deposit.

Recommendations:

It is recommended to state explicitly in SDK and user-facing documentation that disabling fee attachment also disables revocation of pending authorizations.

If owners should retain revocation access during feature disablement, provide a dedicated owner-signed operation that clears the saved builder and factor without requiring replacement-builder accounts. Require correct ownership and store association, a pending order, and zero recorded unpaid builder fee. Preserve all recorded settlement obligations.

Retain the restart guard unless a separate change to the protocol's recovery policy explicitly permits owner revocation during that interval.

GMX: Resolved with PR-457.

Zenith: Verified.

[I-2] Increase trade events leave the builder fee output mint unset

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- programs/store/src/ops/order.rs#L1771-L1808
- programs/store/src/events/trade/data.rs#L232-L243
- programs/store/src/events/trade/data.rs#L350-L370

Description:

The increase fee path introduced in the audited diff routes collected builder fees into the order's final-output escrow. It validates that the mint matches the position's collateral token and records the fee in `transfer_out.final_output_token`.

However, the trade event initializes its corresponding mint field to `DEFAULT_PUBKEY`. The increase path never updates that field, and copying the transfer amounts into the event leaves it unchanged:

```
// TradeData::init
self.final_output_token = DEFAULT_PUBKEY;

// execute_increase_position
transfer_out.transfer_out(false, payable_amount)?;

// TradeData::update_with_transfer_out
self.transfer_out = *transfer_out;
```

A successful market or limit increase with a valid builder-fee checkpoint, initialized final-output escrow and positive collected fee therefore emits a nonzero `TradeEvent.transfer_out.final_output_token` amount alongside an unset `TradeEvent.final_output_token` mint.

Consumers must recover the mint from additional execution or market context. Consumers relying exclusively on the affected field may omit or incorrectly interpret the output, although no concrete downstream misaccounting was established. The actual transfers use validated accounts, so this defect does not establish incorrect fee collection or beneficiary payments.

Recommendations:

It is recommended to populate the trade event's final-output mint from the already validated mint when recording the increase fee transfer:

```
transfer_out.transfer_out(false, payable_amount)?;
event.set_final_output_token(&final_output_token);
```

This uses the existing event field without changing its serialized layout. Clarify in the event documentation that this increase output represents funds reserved for builder-fee settlement.

GMX: Resolved with PR-459.

Zenith: Verified.

[I-3] Builder claim vaults lack a rent recovery path

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- programs/store/src/instructions/builder_fee.rs#L441-L468
- programs/store/src/instructions/builder_fee.rs#L509-L548
- programs/store/src/instructions/builder_fee.rs#L62-L77

Description:

The builder-fee lifecycle introduced in the audited diff uses an associated token account whose token authority is the builder's `UserHeader` PDA. Only the store program can sign for that authority.

`claim_builder_fees` transfers the vault's token balance to an owner-selected destination and leaves the account open. An empty vault returns immediately:

```
let amount = claim_vault.amount;
if amount == 0 {
 return Ok(());
}
```

The program provides no instruction to close the vault or assign a separate close authority. Its rent-exempt deposit therefore remains unrecoverable through the current program after all tokens have been claimed, including when a vault is no longer needed or was created for an unintended mint.

The deposit is an account-level cost: each vault is reused across orders and claims. Ordinary claimable token balances remain withdrawable. Funding account creation does not grant the payer closure authority or determine the beneficiary of a future refund.

Keeping the vault open also preserves a settlement dependency. Checkpointing requires its existence, and settlement expects an existing destination. A zero token balance does not establish that no previously checkpointed order will need the vault.

Recommendations:

It is recommended to document the retained rent-exempt deposit and decide whether recovery should be supported.

If supported, introduce owner-authorized closure of an empty canonical vault using the `UserHeader` PDA signature, with an explicitly defined refund recipient.

Preserve settlement by allowing idempotent recreation of a missing vault immediately before settlement in the same transaction, with an explicit payer. Alternatively, restricting closure while dependent orders remain would require reliable bookkeeping.

Define who bears recurring recreation costs and receives closure proceeds. Atomic recreation restores settlement availability, but repeated claim-and-close cycles can require subsequent settlers to fund the deposit again.

GMX: This is currently the intended design and we do not plan to fix it for now. However, the documentation has been updated in Resolved with PR-463 .

Zenith: Verified.

[I-4] Deferred order closing permits retroactive referral rewards

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- crates/sdk/src/client/ops/exchange/order.rs#L1269-L1287
- programs/store/src/states/order.rs#L715-L726
- programs/store/src/instructions/exchange/order.rs#L899-L950
- programs/store/src/states/user.rs#L220-L232

Description:

Order execution mints GT to the trader and saves the minted amount in `gt_reward` for subsequent referral processing. This amount is recorded even when the trader has no referrer. Closing later determines eligibility using the trader's current referral relationship.

```
// During execution:
self.gt_reward = minted;

// During closing:
let Some(referrer) = self.user.load()?.referral().referrer().copied() else {
 return Ok(());
};
```

A trader who has never registered a referrer can therefore execute an order, register a valid referrer before referral processing, and cause that referrer to receive GT for the earlier execution. Registration does not check for previously executed orders. This contradicts the documented policy: "Rebates apply only to future transactions, and no retroactive rebates will be issued." Referral terms, §3.1.

The referral accounting predates the audited diff. The scoped SDK change exposes this weakness through the normal builder-fee workflow: orders with a nonzero builder fee factor remain open even when automatic closing is requested. Any outstanding builder fee must be settled before closing, creating an interval between execution and referral processing.

The behavior requires a trader without an existing referrer, a positive unprocessed `gt_reward`, registration before referral processing, and a nonzero resulting referral reward. Existing referral relationships cannot be replaced. Successful processing clears `gt_reward`, including when no referrer exists; an order remaining open after partial cleanup does not

necessarily retain an unprocessed reward.

The confirmed impact is retroactive GT issuance inconsistent with referral eligibility policy. The recipient's GT balance increases and may affect future VIP benefits, but material economic loss or an advantage over registering before trading has not been established. The trader's original fees and discount remain unchanged, and the reward cannot be repeatedly consumed.

Closing also uses the referrer's current rank and configured reward factor. The documentation does not clearly require execution-time rate locking, so that behavior is not treated as an additional demonstrated violation.

Recommendations:

It is recommended to record a deferred referral reward only when the trader has a referrer at execution:

```
self.gt_reward = if user.referral().referrer().is_some() {
 minted
} else {
 0
};
```

Because an established referrer cannot be replaced, this prevents retroactive eligibility without adding a beneficiary field. Preserve the trader's own GT minting, existing beneficiary validation, and one-time reward consumption.

Define handling for already-executed orders separately, since their saved reward amount does not establish eligibility at execution.

If execution-time reward rates are also intended, snapshot the applicable factor or final reward amount. Recording rank alone would still allow subsequent configuration changes to affect payment.

GMX: Acknowledged. This will be fixed in the future version.

Zenith: Acknowledged.